Bonus — Progetto Finale Cybersecurity

Nov 10, 2026 · @Danilo Noccioli

Introduzione

Questo documento raccoglie i cinque bonus facoltativi del progetto finale; le sei challenge sono documentate nei loro paper (cartella `docs/` del repository). Oltre ai bonus, una sezione a parte descrive l'aggiornamento delle dipendenze vulnerabili, fatto con tre strumenti diversi.

Bonus	Tema	OWASP	Esito
1	Rate limiter sul login di Fortify	A07 Identification and Authentication Failures	5 tentativi al minuto per coppia email+IP, poi 429
2	Clickjacking	A05 Security Misconfiguration	`X-Frame-Options: DENY` e CSP `frame-ancestors 'none'`
3	Ricerca full-text con Laravel Scout	A03 Injection	ricerca tramite indice dedicato, senza SQL costruito con l'input
4	Scansione OWASP ZAP	A05, A06	alert da 12 a 11, classificati uno per uno
5	Autorizzazione con le Policy	A01 Broken Access Control	403 su accesso a oggetti altrui e su cambio ruolo non autorizzato, con log

Bonus 1 — Rate limiter sul login di Fortify

Il login accetta al massimo 5 tentativi al minuto per ogni coppia email+IP: dal sesto in poi il server risponde 429 e scrive l'evento nel log come errore. Previene il brute force sulle credenziali (A07 Identification and Authentication Failures, CWE-307) ed è il terzo livello della Challenge 1.

Implementazione

Il limiter si chiama `login` ed è definito in `app/Providers/FortifyServiceProvider.php`. La chiave di conteggio unisce l'email (in minuscolo) e l'IP, quindi i tentativi sbagliati su un account non bloccano gli altri utenti.

Verifica

Dopo 5 tentativi con password errata il browser mostra *429 Too Many Requests*. Il log (orari in UTC) mostra i cinque login falliti e, alla sesta richiesta, il blocco:

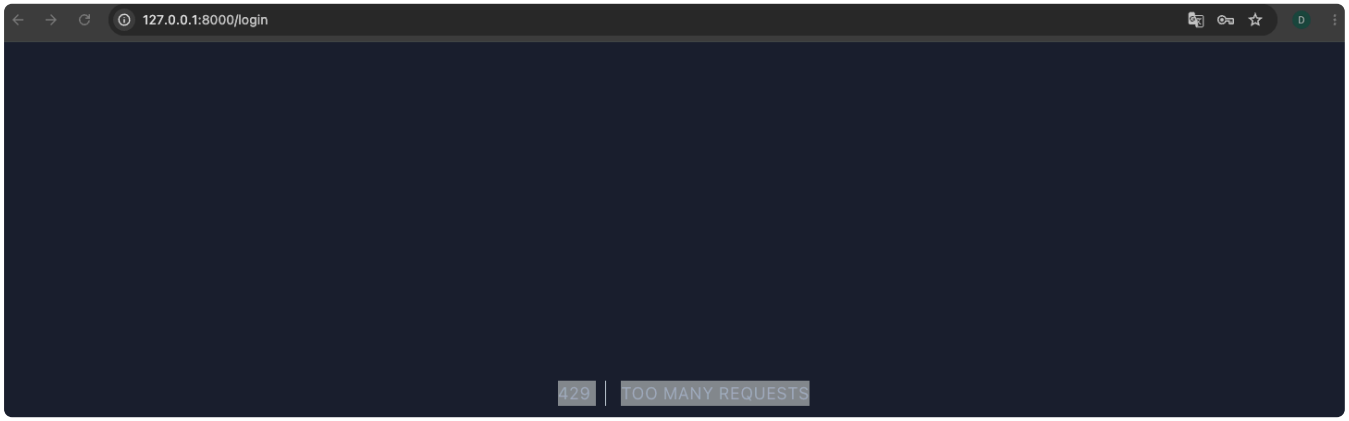


Figura 1 — Dopo cinque tentativi falliti il login risponde 429 Too Many Requests.

```
[2026-11-04 11:31:25] local.WARNING: User login failed
{"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:27] local.WARNING: User login failed
{"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:29] local.WARNING: User login failed
{"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:30] local.WARNING: User login failed
{"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:32] local.WARNING: User login failed
{"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:33] local.ERROR: User lockout: too many failed login
attempts {"email":"user@aulab.it","ip":"127.0.0.1"}
```

```
❯ grep -E "local\.(INFO|WARNING|ERROR)" ~/cyberblog-logs/laravel.log
[2026-11-04 11:27:11] local.INFO: User logged out {"user_id":7,"ip":"127.0.0.1"}
[2026-11-04 11:27:17] local.INFO: User logged in {"user_id":8,"ip":"127.0.0.1"}
[2026-11-04 11:27:37] local.INFO: User promoted to revisor {"actor_id":6,"target_id":8,"ip":"127.0.0.1","before":null,"after":true}
[2026-11-04 11:31:15] local.INFO: User logged out {"user_id":8,"ip":"127.0.0.1"}
[2026-11-04 11:31:25] local.WARNING: User login failed {"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:27] local.WARNING: User login failed {"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:29] local.WARNING: User login failed {"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:30] local.WARNING: User login failed {"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:32] local.WARNING: User login failed {"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:31:33] local.ERROR: User lockout: too many failed login attempts {"email":"user@aulab.it","ip":"127.0.0.1"}
[2026-11-04 11:32:10] local.INFO: User logged in {"user_id":4,"ip":"127.0.0.1"}
[2026-11-04 11:32:20] local.WARNING: Unauthorized access attempt to admin route {"user_id":4,"ip":"127.0.0.1","route":"admin/dashboard"}
```

Figura 2 — Estratto di laravel.log: i cinque login falliti, il lockout registrato come errore e un accesso negato a una rotta admin.

Hardening delle sessioni (iniziativa personale)

Nello stesso ambito A07 ho rinforzato la configurazione delle sessioni nel `.env`:

Impostazione	Effetto
`SESSION_DRIVER=database`	sessioni nel database, revocabili lato server in caso di compromissione di un account
`SESSION_ENCRYPT=true`	dati di sessione cifrati, meno rischio se qualcuno accede allo storage
`SESSION_LIFETIME=120`	scadenza dopo 120 minuti di inattività, per ridurre la finestra di una sessione rubata
`SESSION_EXPIRE_ON_CLOSE=true`	la sessione termina alla chiusura del browser

Bonus 2 — Clickjacking

Ogni risposta dell'applicazione vieta di essere caricata in un frame: il browser rifiuta di mostrare il sito dentro un `<iframe>` di un'altra pagina, quindi un attaccante non può sovrapporre elementi invisibili per far cliccare la vittima a sua insaputa (A05 Security Misconfiguration).

Il rischio

Nel clickjacking il sito viene caricato in un `<iframe>` trasparente su una pagina dell'attaccante: la vittima crede di cliccare sulla pagina che vede, ma il click arriva a un elemento del sito reale, per esempio un pulsante di cambio ruolo o di cancellazione.

Difesa

Il middleware `SecurityHeaders`, registrato globalmente in `bootstrap/app.php`, aggiunge su ogni risposta due controlli indipendenti:

`X-Frame-Options: DENY` è l'header storico; `frame-ancestors 'none'` è la direttiva CSP che lo sostituisce nei browser moderni. Tenerli entrambi copre anche i browser più vecchi. `frame-src 'none'` limita invece i frame che la nostra pagina può incorporare, ed è un controllo diverso: riguarda i frame dentro il sito, non il sito dentro un frame.

Verifica

La direttiva `frame-ancestors` è stata aggiunta in un secondo momento, dopo la prima scansione OWASP ZAP (Bonus 4): il report "prima" segnalava la sua assenza dalla CSP, il report "dopo" non contiene più quell'alert.

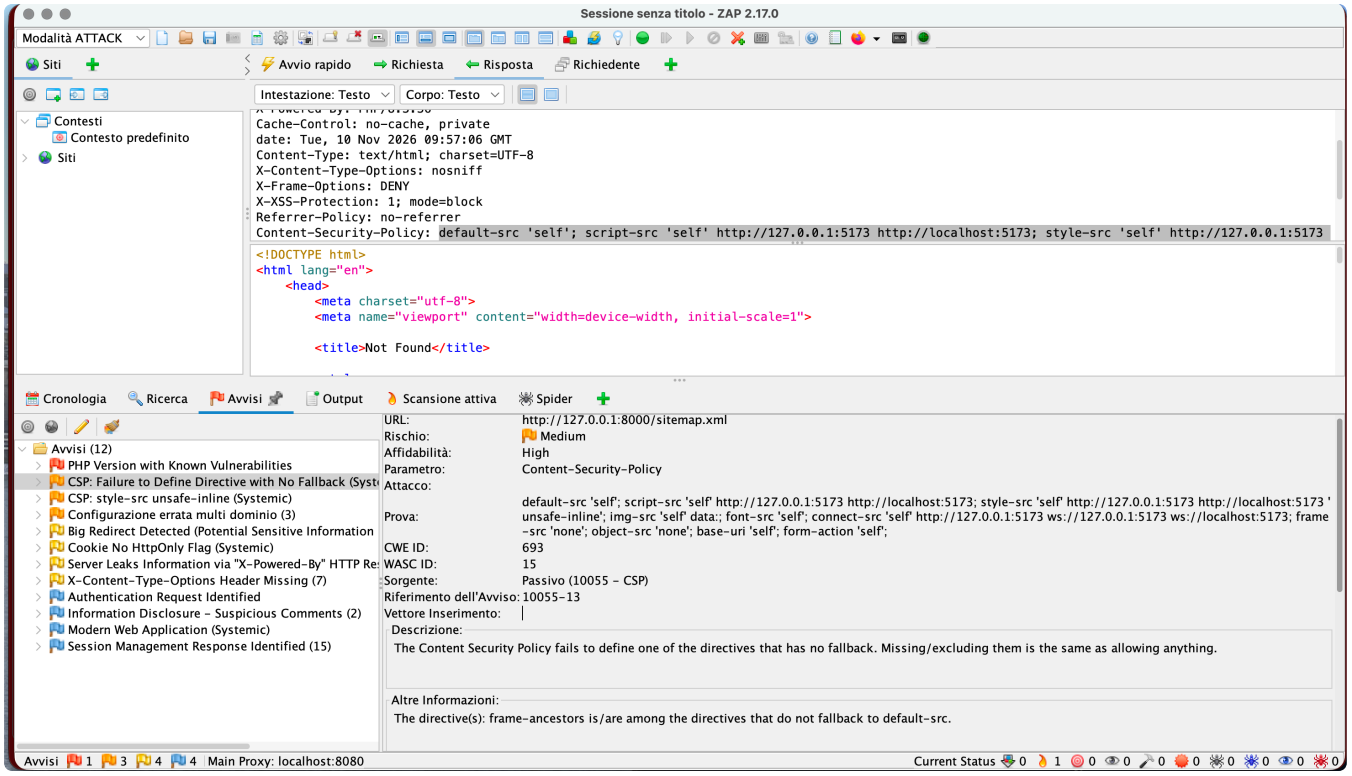


Figura 3 — Scansione ZAP prima della correzione: l'alert segnala che la CSP non definisce frame-ancestors.

Nota: la dimostrazione dell'attacco con una pagina in `<iframe>` non è documentata in questo paper; la verifica si basa sugli header inviati e sul confronto tra i due report ZAP. Il middleware imposta ancora `X-XSS-Protection: 1; mode=block`, un header che i browser moderni ignorano e che le linee guida correnti consigliano di impostare a `0`.

Bonus 3 — Ricerca full-text con Laravel Scout

La ricerca degli articoli passa da Laravel Scout con il driver TNTSearch: il testo cercato dall'utente viene affidato a un motore di ricerca su un indice dedicato e non entra mai in una query SQL costruita a mano, quindi non c'è modo di iniettarvi SQL (A03 Injection). Scout e TNTSearch erano già presenti nel progetto di partenza; qui sono stati studiati e verificati.

Come funziona

Il modello `Article` usa il trait `Searchable` e decide quali campi finiscono nell'indice con `toSearchableArray()`:

Il controller riceve il testo e lo passa a Scout; l'unico filtro aggiunto riguarda gli articoli accettati dai revisori:

Il driver è scelto in `config/scout.php` (`SCOUT_DRIVER`, predefinito `tntsearch`).

Perché è sicura contro la SQL injection

In una ricerca tradizionale con `LIKE` il testo dell'utente finisce dentro una query SQL: se viene concatenato, un input come `' OR 1=1 --` cambia il significato della query. Con Scout il testo viene interpretato dal motore di ricerca come semplici termini da cercare nell'indice, e il database non lo vede mai come codice.

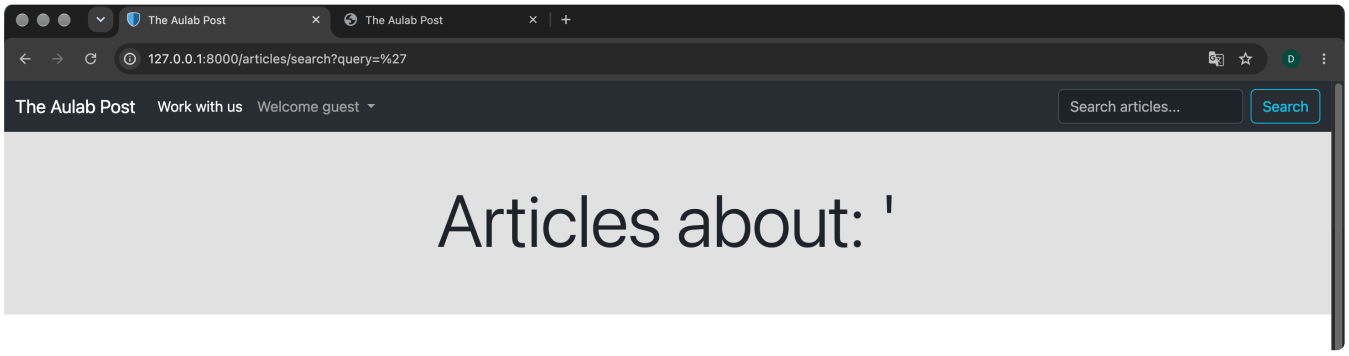


Figura 4 — Ricerca con il carattere ': la pagina risponde normalmente, nessun errore SQL.

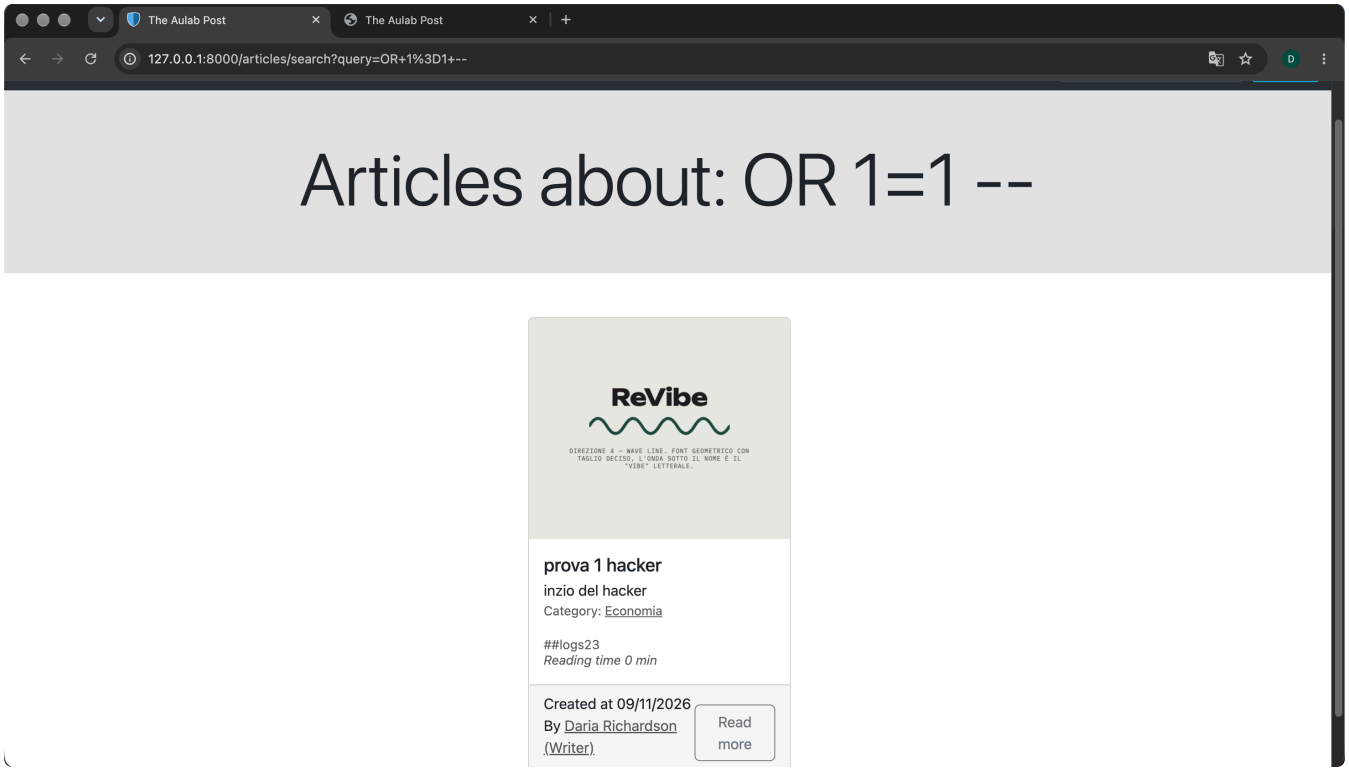


Figura 5 — Ricerca con OR 1=1 --: viene trattato come testo da cercare nell'indice.

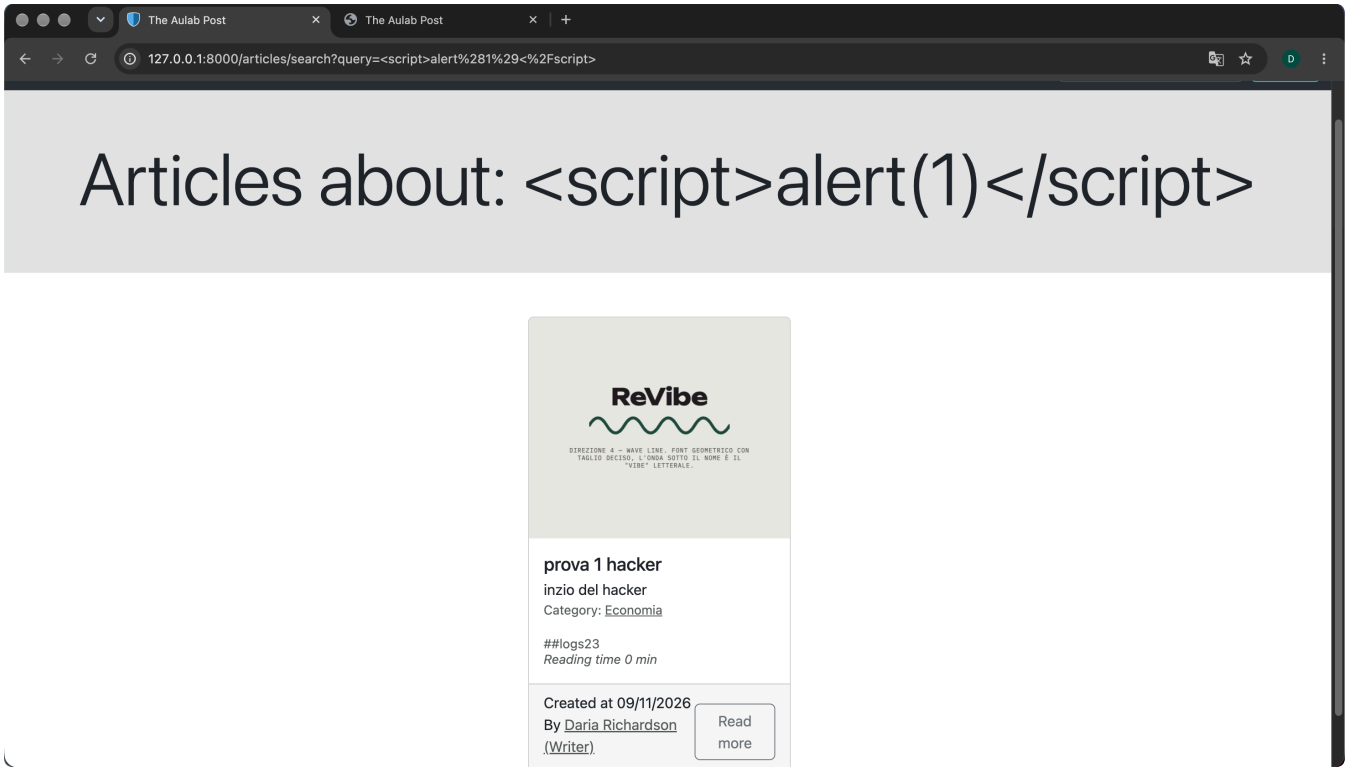


Figura 6 — Ricerca con un tag script: il testo è mostrato come testo e non viene eseguito.

Protezione dall'abuso

La rotta `/articles/search` è pubblica e costosa da servire, quindi è coperta dai due livelli della Challenge 1: `block.suspicious` (blocco temporaneo dell'IP) e `throttle:20,1` (20 richieste al minuto).

Dopo l'aggiornamento delle dipendenze

L'upgrade a Laravel 12 ha portato il driver da v14.0.0 a v16.2.0 e `teamtnt/tntsearch` da 4.2.1 a 5.3.0. Dopo il cambio di versione l'indice è stato ricostruito con:

```
php artisan scout:import "App\Models\Article"
```

Il comando ha reindicizzato tutti gli articoli, fino all'ID 8.

Bonus 4 — Scansione con OWASP ZAP

Una scansione automatica con OWASP ZAP 2.17.0 ha dato 12 alert prima delle correzioni e 11 dopo: l'unico alert risolto è la direttiva `frame-ancestors` mancante dalla CSP (Bonus 2), e gli altri undici sono stati valutati uno per uno.

Metodo

ZAP è uno scanner DAST (*Dynamic Application Security Testing*): analizza l'applicazione in esecuzione, non il codice. La stessa scansione è stata ripetuta due volte, prima e dopo le modifiche di sicurezza (tra cui la CSP con `frame-ancestors`), e i due report sono stati confrontati. I report HTML contengono valori di cookie della sessione di prova e per questo non sono incorporati nel paper.

Report	Alert
Prima delle correzioni	12
Dopo le correzioni	11

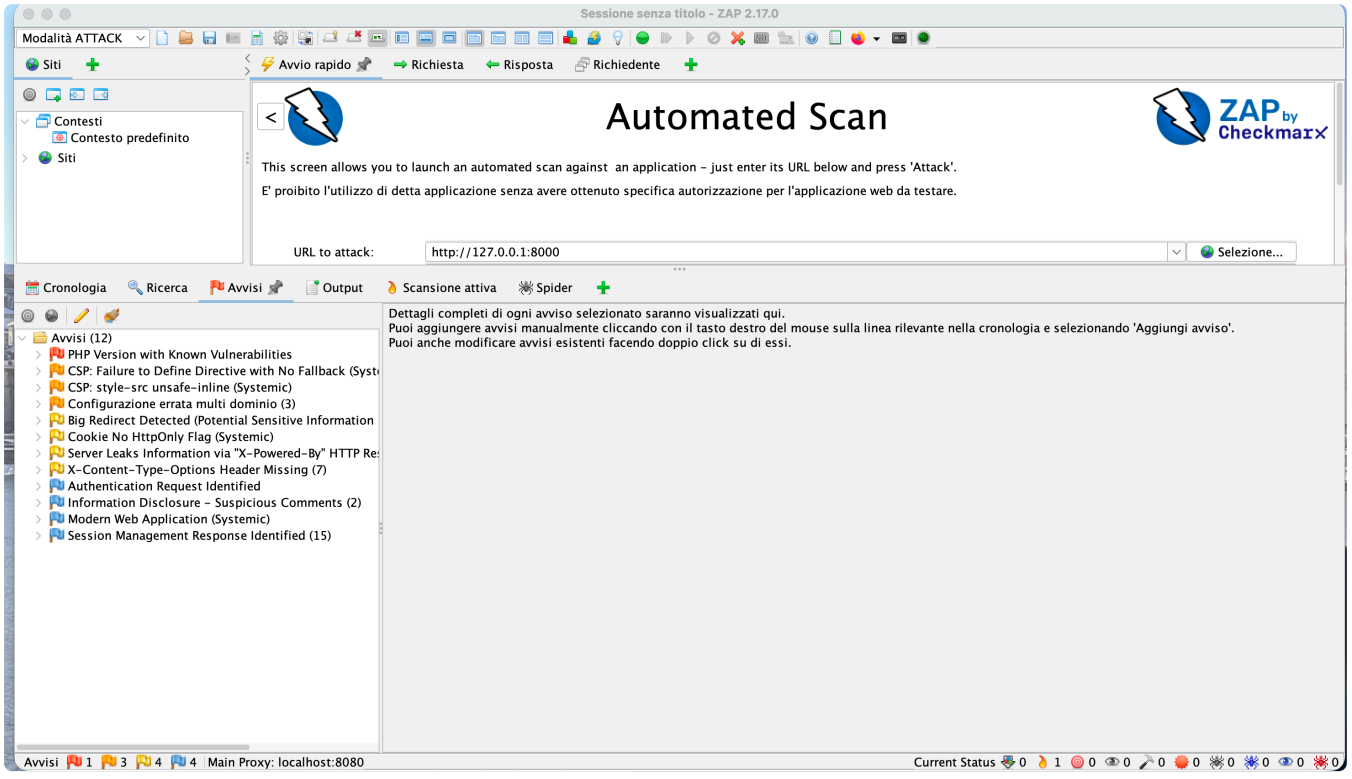


Figura 7 — Scansione ZAP prima delle correzioni: 12 alert.

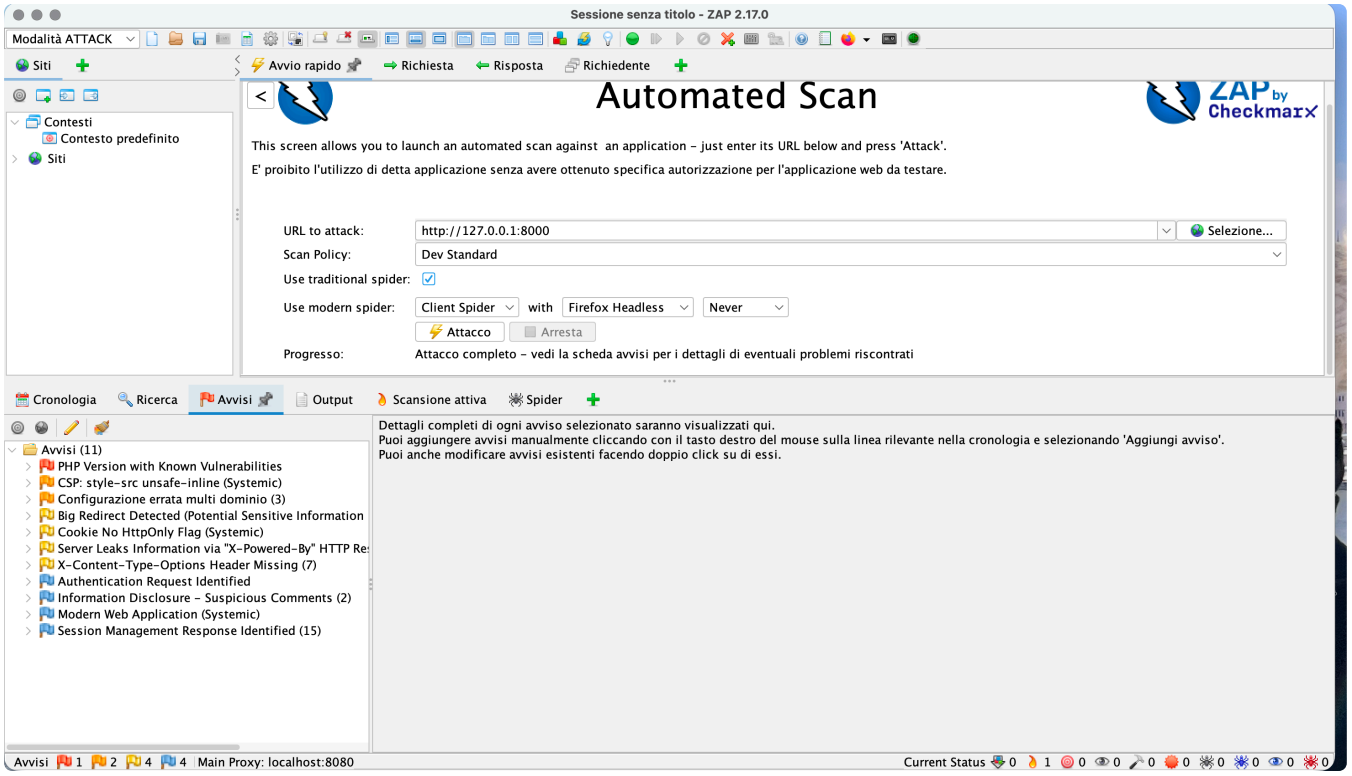


Figura 8 — Scansione ZAP dopo le correzioni: 11 alert.

Alert residui e valutazione

Alert	Valutazione	Decisione
Segnalazione legata a PHP	probabile falso positivo	nessuna azione
CSP con `style-src 'unsafe-inline'`	compromesso consapevole: stili inline di Bootstrap e Blade, non legati a input utente (vedi Challenge 5)	accettato
CORS verso il server Vite	presente solo con il server di sviluppo	accettato, assente in produzione
Cookie `XSRF-TOKEN` senza `HttpOnly`	atteso: il JavaScript deve poterlo leggere per inviarlo nelle richieste	accettato
Header `X-Powered-By`	reale: espone il linguaggio del server	correggibile con `expose_php = Off` in `php.ini`, facoltativo
`X-Content-Type-Options` assente su file statici (es. `robots.txt`)	reale ma di basso impatto: i file statici non passano dal middleware di Laravel	facoltativo, si risolve sul web server

L'esercizio mostra due cose: uno scanner automatico segnala anche comportamenti voluti, e ogni alert va valutato nel contesto prima di decidere se correggere, accettare o scartare.

Bonus 5 — Autorizzazione con le Policy

Le regole su chi può modificare o cancellare un articolo e su chi può assegnare un ruolo sono ora concentrate in due Policy di Laravel, controllate dal server a ogni richiesta: un writer non può più toccare gli articoli di un altro writer, e ogni tentativo negato risponde 403 e finisce nel log (A01 Broken Access Control, API5).

Attacco: accesso a oggetti altrui (BOLA/IDOR)

Il controllo sul ruolo (`writer`) c'era, ma non quello sulla proprietà dell'oggetto: qualunque writer loggato poteva chiamare le rotte di modifica e cancellazione con l'id o lo slug di un articolo scritto da un altro.

Prova: l'utente 2 ha creato un articolo di test; dalla console del browser, loggato come utente 3, è stata inviata una richiesta `DELETE` con `_method` e token CSRF verso `/articles/destroy/{articolo}` . La risposta è stata 200 e l'articolo è stato cancellato; il log del server mostra l'utente 3 che elimina l'articolo dell'utente 2.

Mitigazione

1 — **`ArticlePolicy`** : modifica e cancellazione sono permesse all'autore dell'articolo o a un admin.

2 — **`UserPolicy`** : solo un admin può assegnare un ruolo, e solo se l'utente ha una richiesta in attesa (il flag del ruolo vale `null`).

3 — Controllo nei controller, come prima istruzione di ogni metodo coinvolto. `Gate::authorize()` lancia l'eccezione che Laravel trasforma in risposta 403:

Il controllo manuale che esisteva in `edit()` (confronto tra utente e proprietario con redirect) è stato sostituito dalla Policy. Le chiamate a `Gate::authorize` stanno solo nei controller: dentro la Policy provocherebbero una ricorsione infinita.

4 — Log degli accessi negati. Laravel considera `AuthorizationException` tra le eccezioni da non riportare, quindi per averla nel log serve toglierla dall'elenco in `bootstrap/app.php`

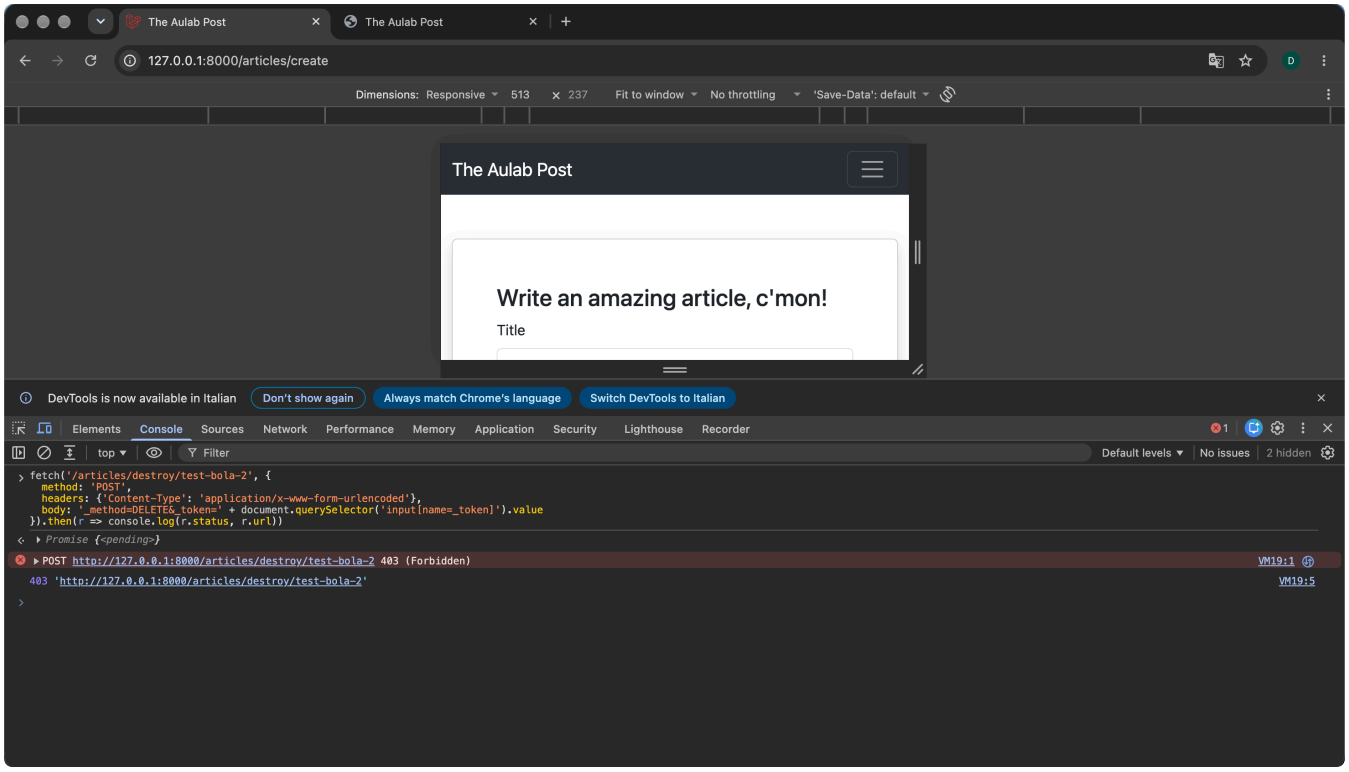


Figura 9 — Stessa richiesta DELETE della prova, ora negata con 403 Forbidden.

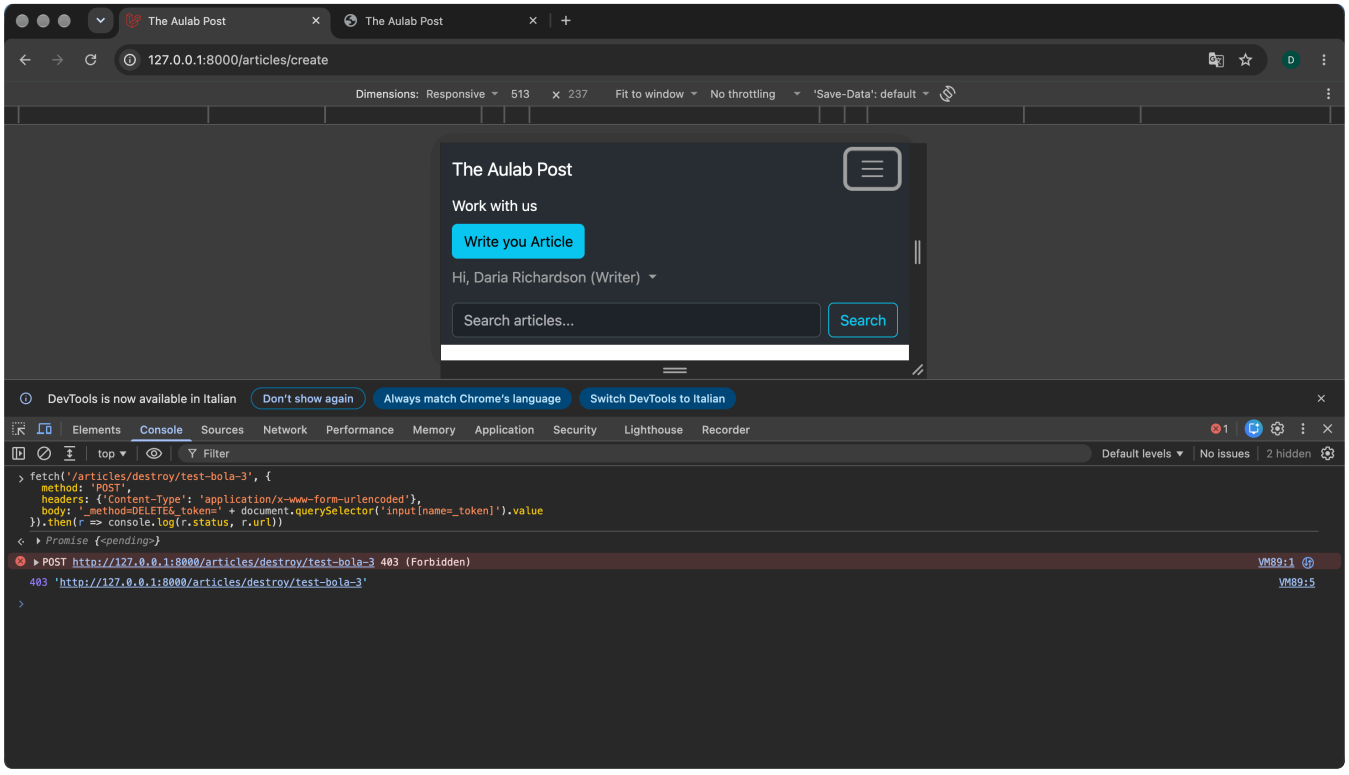


Figura 10 — Ripetizione della prova su un secondo articolo: 403.

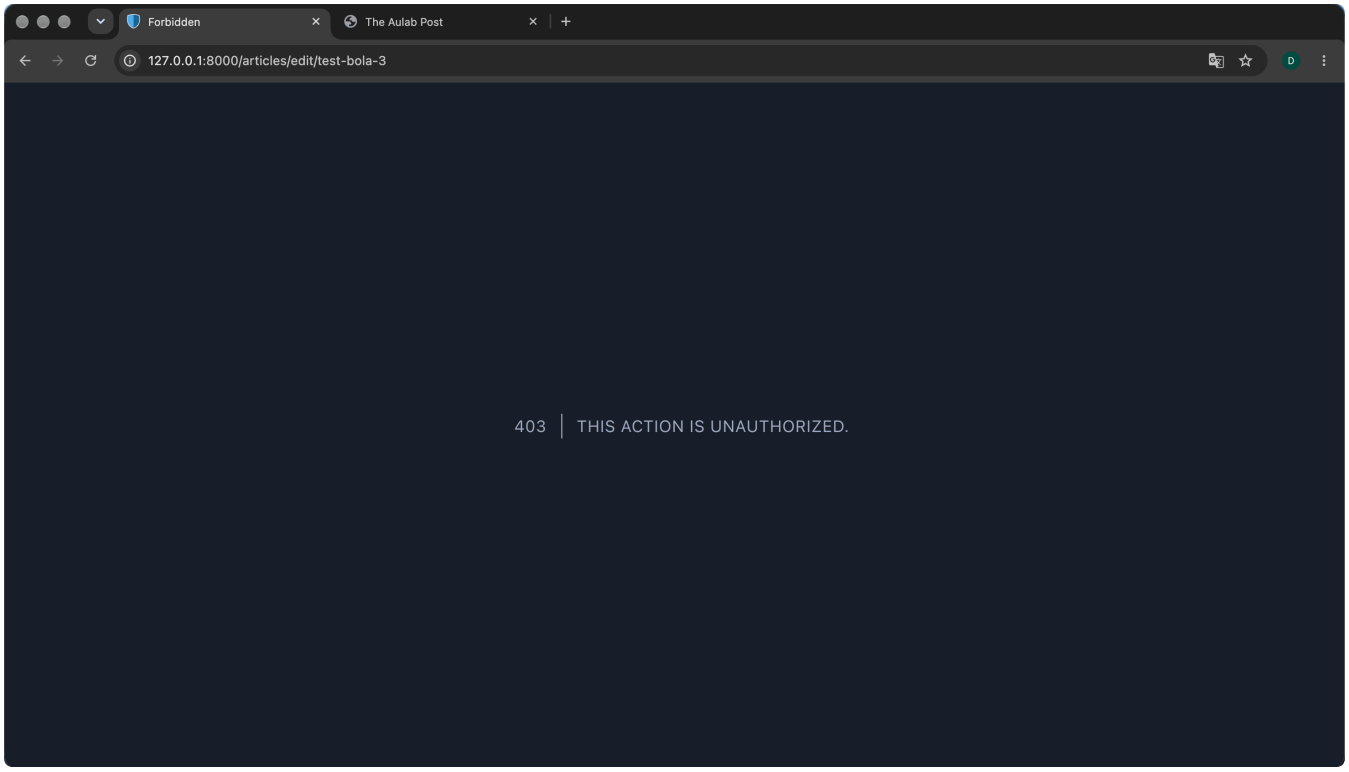


Figura 11 — Apertura della pagina di modifica di un articolo altrui: 403 This action is unauthorized.

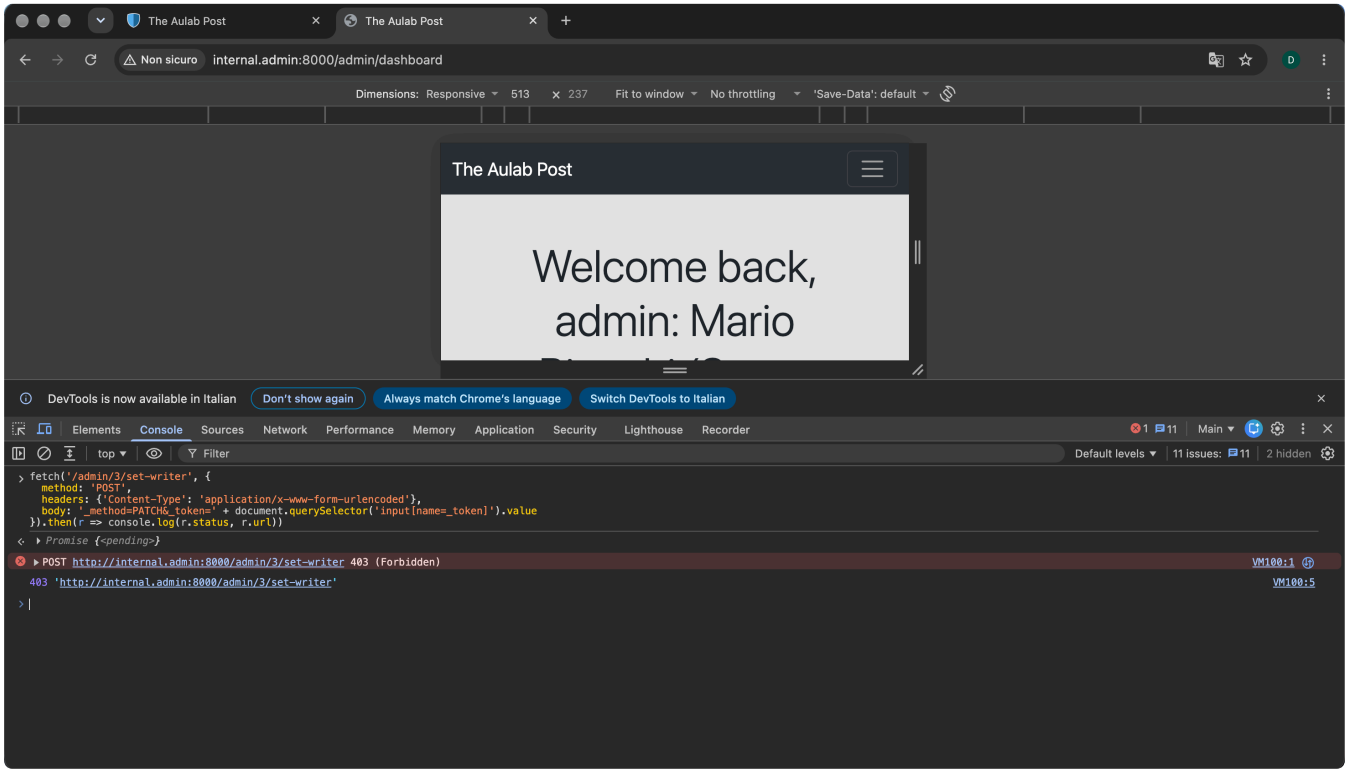


Figura 12 — Richiesta di cambio ruolo da un utente non admin: 403.

Componenti vulnerabili (A06) — audit e aggiornamento

Tre strumenti diversi hanno controllato le dipendenze e l'aggiornamento a Laravel 12 e Vite 7 le ha portate da 61 avvisi sul lato PHP e 10 vulnerabilità sul lato JavaScript a zero in entrambi i casi (A06 Vulnerable and Outdated Components).

Controllo	Prima	Dopo
`composer audit`	61 avvisi in 17 pacchetti	nessun avviso
`npm audit`	10 vulnerabilità (1 critica, 7 alte, 2 moderate)	0
Snyk, Composer (474 dipendenze, incluse le dev)	non eseguito	nessun percorso vulnerabile
Snyk, npm del progetto	8 problemi su Vite 5.4.21 ed esbuild (1 critico)	95 dipendenze, nessun problema

Perché serviva Laravel 12

L'audit di Composer segnalava il framework stesso: Laravel 11.4.0 era in una versione con avvisi di sicurezza, e Laravel 11 non riceve più correzioni di sicurezza dal 12 marzo 2026 (Laravel 12 le riceve fino al 24 febbraio 2027). Una semplice `composer update` dentro `^11.0` era bloccata perché tutte le versioni di Laravel 11 erano interessate dagli avvisi, quindi l'unica correzione era il salto di versione maggiore.

Prima di cominciare sono stati fatti un backup del database e un commit del lavoro dei bonus, e l'aggiornamento è stato svolto su un branch separato (`upgrade-laravel-12`).

Lato PHP: Laravel 12

In `composer.json` `laravel/framework` è passato da `^11.0` a `^12.0` . Un secondo ostacolo è stato il driver di ricerca: la versione 14 di `teamtnt/laravel-scout-tntsearch-driver` non supporta `illuminate` 12, quindi il vincolo è salito a `^16.0` .